Triggers

Triggers

- Triggers are stored programs, which are automatically executed or fired when some events occur.
- A **database manipulation (DML)** statement (DELETE, INSERT, or UPDATE)
- A **database definition (DDL)** statement (CREATE, ALTER, or DROP).
- A **database operation** (SERVERERROR, LOGON, LOGOFF, STARTUP, or SHUTDOWN).

Benefits of Triggers

- Generating some **derived column values** automatically
- Enforcing **referential integrity**
- Event logging and storing information on table access
- Auditing**(security purposes)**
- Imposing security authorizations
- Preventing invalid transactions

Syntax

```
CREATE [OR REPLACE ] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF }
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW]
WHEN (condition)
DECLARE
   Declaration-statements
BEGIN
   Executable-statements
EXCEPTION
   Exception-handling-statements
END;
```

Types of Triggers

- **Statement-level triggers:** Statement-level triggers are fired once for collection of rows affected by SQL statement. They respond to a single SQL statement and can operate on multiple rows at a time.
- For example, if a **single SQL INSERT** statement **adds multiple rows** to a table, a statement-level trigger associated with that table will **fire only once** for the entire statement, regardless of the number of rows affected.
- **Row-level triggers:** Row-level triggers are fired once for each row affected by an SQL statement. They respond to individual rows being affected by DML (Data Manipulation Language) statements like INSERT, UPDATE, and DELETE. This means that if an SQL statement affects multiple rows, **a row-level trigger will be invoked once for each affected row.**

Trigger Parts

- Event

  A change to data base that activates the trigger

- Restriction

  A trigger restriction specifies a Boolean (logical) expression that must be TRUE for the trigger to fire

- Action

  A procedure that is executed when the trigger is activated.
  Similar to stored procedures, a trigger action can contain PL/SQL statements

Trigger: Events

- **Three event types**
  - Insert
  - Update
  - Delete
- **Two triggering times**
  - Before the event
  - After the event
- **Two granularities**
  - Execute for each row
  - Execute for each statement

1) Trigger: Event

Trigger name

```
Create Trigger <name>
Before|After     Insert|Update|Delete  ON <tablename>

....
```

That is the event

- Example

```
Create Trigger ABC
Before  Insert On Students
....
```

This trigger is activated when an insert statement is issued, but before the new record is inserted

```
Create Trigger XYZ
After Update On Students
....
```

This trigger is activated when an update statement is issued and after the update is executed

2) Trigger: Condition

- This component is **optional**

```
Create Trigger <name>
Before| After      Insert| Update| Delete On <tableName>
For Each Row | For Each Statement

When <condition>
...
```

That is the condition

If the employee salary > 150,000 then some actions will be taken

```
Create Trigger EmpSal
After Insert or Update On Employee
For Each Row
When (new.salary >150,000)
...
```

3) Trigger: Action

- **Action depends on what you want to do, e.g.:**
  - Check certain values
  - Fill in some values
  - Inserts/deletes/updates other records
  - Check that some business constraints are satisfied
  - Commit (approve the transaction) or roll back (cancel the transaction)
- **In the action, you may want to reference:**
  - The new values of inserted or updated records ***(:new)***
  - The old values of deleted or updated records ***(:old)***

Trigger: Referencing Values

- **In the action, you may want to reference:**
  - The new values of inserted or updated records ***(:new)***
  - The old values of deleted or updated records ***(:old)***



Trigger Timings

- Before Trigger

  Execute the trigger action before the triggering statement. Eliminate unnecessary processing of the triggering statement.

- After Trigger

  AFTER triggers are used when you want the triggering statement to complete before executing the trigger action

Syntax for creating triggers

- CREATE [OR REPLACE] TRIGGER trigger_name
- {BEFORE | AFTER | INSTEAD OF }
- {INSERT [OR] | UPDATE [OR] | DELETE}
- [OF col_name] ON table_name
- [REFERENCING OLD AS o NEW AS n]
- [FOR EACH ROW]
- WHEN (condition)
- DECLAREDeclaration-statements
- BEGIN Executable-statements
- EXCEPTION Exception-handling-statements
- END;

Example

- For example, the following trigger is fired only when AMOUNT is more than 1000.

create or replace trigger ..

before insert on payments

for each row

when :new.amount > 1000

Example 1

If the employee salary increased by more than 10%, make sure the 'rank' field is not empty and its value has changed, otherwise reject the update

If the trigger exists, then drop it first

```
Create or Replace Trigger EmpSal
Before Update On Employee
For Each Row
Begin
      IF (:new.salary > (:old.salary * 1.1)) Then
          IF (:new.rank is null or  :new.rank = :old.rank) Then
               RAISE_APPLICATION_ERROR(-20004, 'rank field not correct');
          End IF;
      End IF;
End;
/
```

Compare the old and new salaries

Make sure to have the "/" to run the command

Example 2

If the employee salary increased by more than 10%, then increment the rank field by 1.

In the case of **Update** event only, we can specify which columns

```
Create or Replace Trigger EmpSal
Before Update Of salary On Employee
For Each Row
Begin
      IF (:new.salary > (:old.salary * 1.1)) Then
          :new.rank := :old.rank + 1;
      End IF;
End;
/
```

We changed the new value of ***rank*** field

The assignment operator has ":"

Example 3: Using Temp Variable

If the newly inserted record in employee has null hireDate field, fill it in with the current date

```
Create Trigger EmpDate
Before Insert On Employee
For Each Row
Declare
    temp date;
Begin
    Select sysdate into temp from dual;
    IF (:new.hireDate is null) Then
        :new.hireDate := temp;
    End IF;
End;
/
```

Since we need to change values, then it should be "Before" event

Declare section to define variables

Oracle way to select the current date

Updating the new value of hireDate before inserting it

Dropping a Trigger

- When a trigger is no longer needed it can be dropped using DROP TRIGGER command .

```
drop trigger triggername;
```